Dependency Injection

A class should not create what it needs. It should receive it.

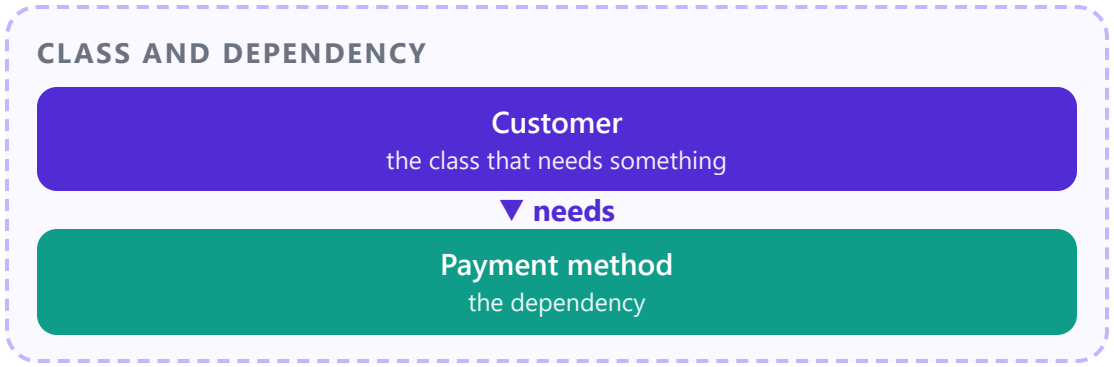
1 The problem

When you write `new` inside a class, you tie that class to that type forever. You want to change it? You must edit the class. The solution is **Dependency Injection**, which ASP.NET Core relies on for almost everything.

2 What is a Dependency?

If class A needs class B to do its work, then B is a **Dependency** of A.

Simple example: a `Customer` needs a payment method to buy something. So the payment method is the dependency of the customer.



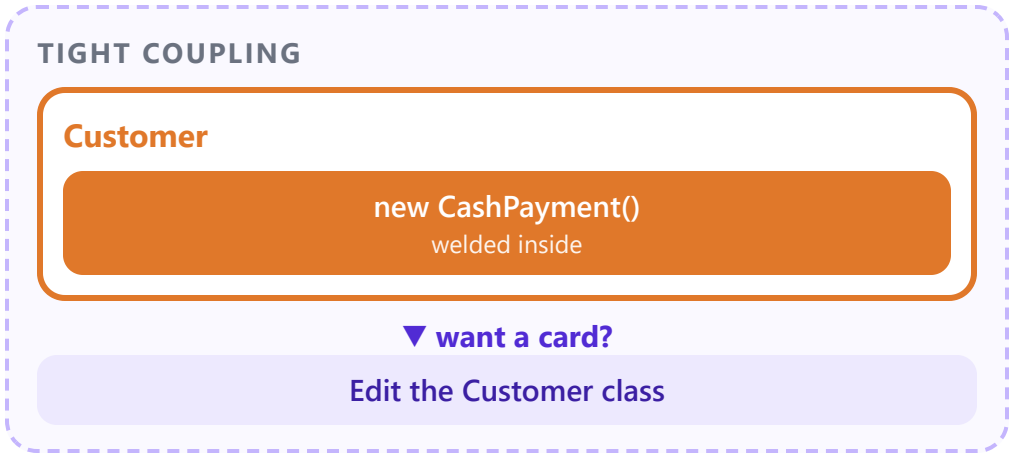
Without DI

The customer creates the payment method by itself

```
public class CashPayment
{
    public string Pay() => "Paid with cash";
}

public class Customer
{
    // the class creates it by itself
    private readonly CashPayment _payment = new CashPayment();

    public string Buy()
    {
        return _payment.Pay() + " - purchase completed";
    }
}
```



The problem: Tight Coupling. The customer is tied to cash payment only. Want to pay by card? You must open the `Customer` class and edit it. It is like a person who has one fixed way of paying and cannot change it.

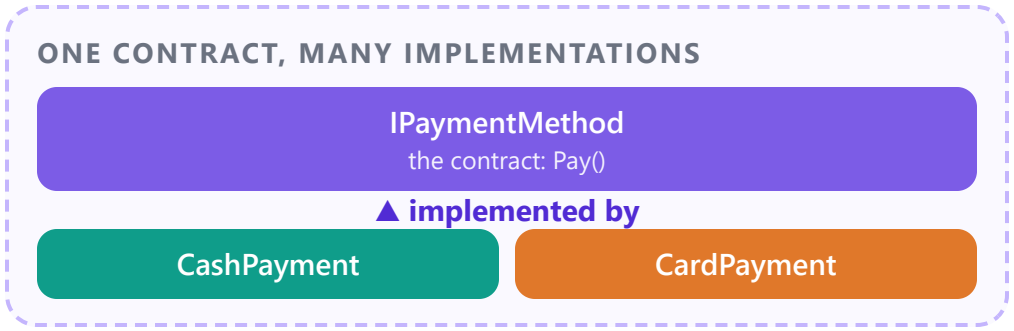
With DI: step 1, an interface

A contract that any payment method must follow

```
public interface IPaymentMethod
{
    string Pay();
}

public class CashPayment : IPaymentMethod
{
    public string Pay() => "Paid with cash";
}

public class CardPayment : IPaymentMethod
{
    public string Pay() => "Paid with card";
}
```



> The interface says: **any payment method must have a `Pay` method**. Now `CashPayment` and `CardPayment` can replace each other.

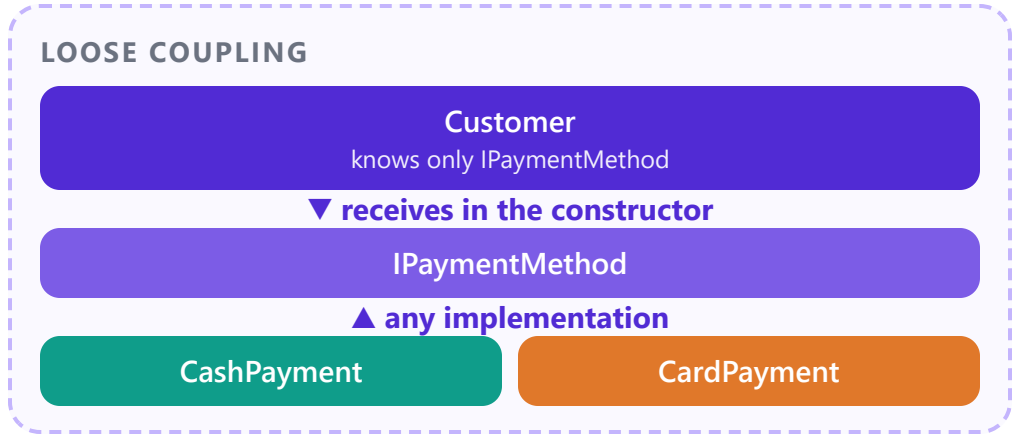
With DI: step 2, the constructor

The customer receives the payment method and does not create it

```
public class Customer
{
    private readonly IPaymentMethod _payment;

    public Customer(IPaymentMethod payment)
    {
        _payment = payment;
    }

    public string Buy()
    {
        return _payment.Pay() + " - purchase completed";
    }
}
```



> The customer no longer knows which payment method it uses. It only knows `IPaymentMethod`. Any class that implements the interface can be given to it. This is **Dependency Injection**.

The Dependency Inversion Principle

The letter D in SOLID

> **High-level modules should not depend on low-level modules. Both should depend on abstractions.**
>
> **Abstractions should not depend on details. Details should depend on abstractions.**

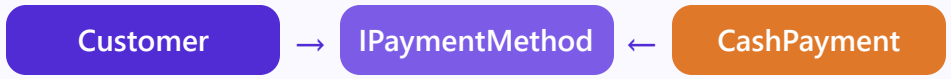
In the principle	In our example
High-level module the app logic: *what* happens	`Customer` the buying logic
Low-level module the details: *how* it happens	`CashPayment`, `CardPayment` how the payment is done
Abstraction separates them	`IPaymentMethod`

BEFORE THE PRINCIPLE



the high-level module depends on the detail

AFTER THE PRINCIPLE



both depend on the abstraction

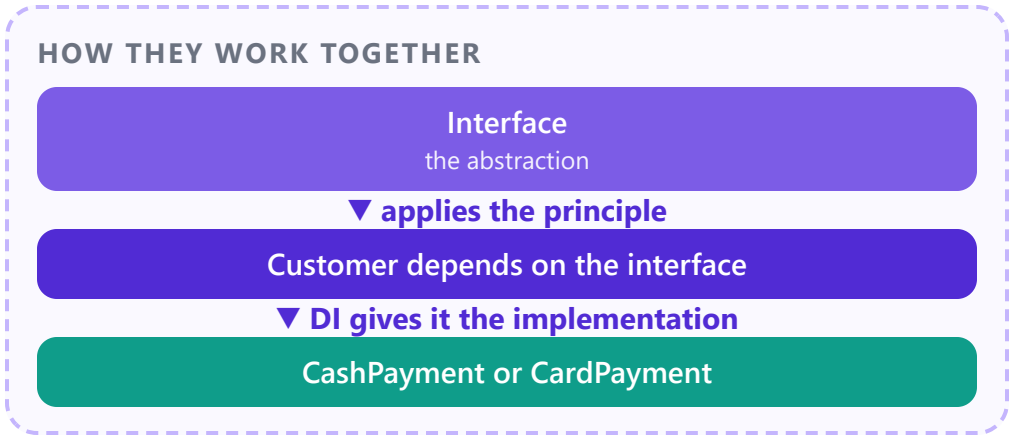
Why "inversion"? The direction of the dependency flips. `CashPayment` now depends on the interface that the high-level module needs, and not the other way around.

Principle or technique?

Dependency Inversion and Dependency Injection are not the same thing

	What it is	It answers
Dependency Inversion	A design **principle**	**What** to do: depend on abstractions
Dependency Injection	An implementation **technique**	**How** to apply it: pass what the class needs from outside

> We use an **interface** to apply the principle, and we use **DI** to give the class the right implementation.



What is Dependency Injection?

A class receives what it needs from outside

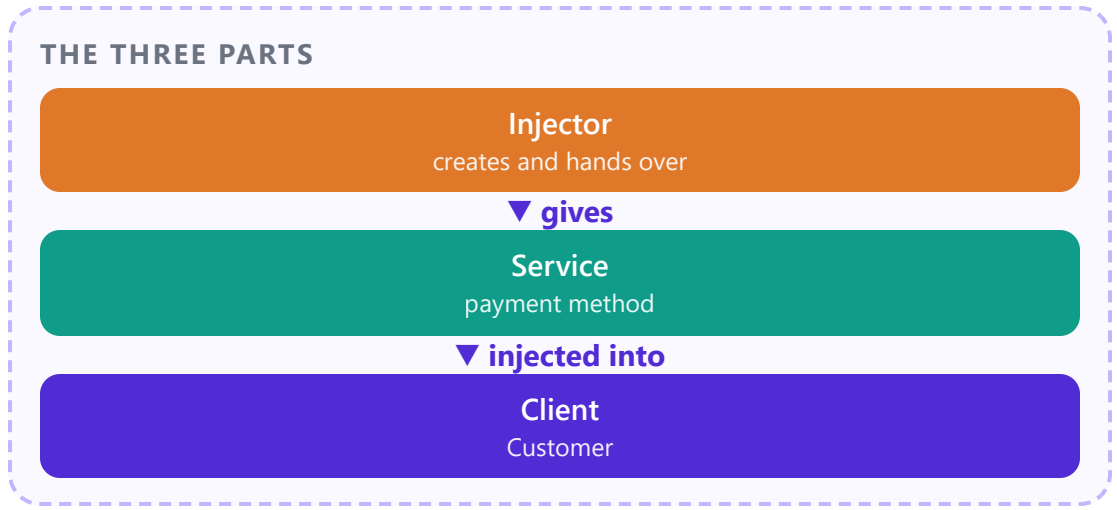
The definition

A technique where a class **receives what it needs from outside** instead of creating it itself.

In our example: instead of the customer creating its payment method, the payment method is **given** to it.

Part	In our example
Service (what is given)	The payment method `IPaymentMethod`
Client (who needs it)	The customer `Customer`
Injector (who gives it)	The code that creates the payment method and passes it to the customer

> The most common way to inject is through the **constructor**, as we did in `Customer` .



Experiment 1: pass it by hand

Put the classes in a folder named `Services`

PROGRAM.CS

```
using ProjectStructure.Services;

var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

app.MapGet("/buy", () =>
{
    IPaymentMethod payment = new CashPayment();
    Customer customer = new Customer(payment);
    return customer.Buy();
});

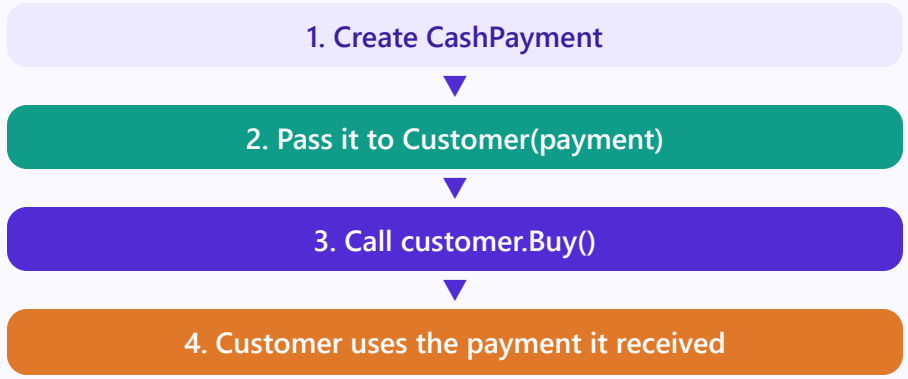
app.Run();
```

OPEN /BUY

```
Paid with cash - purchase completed
```

- We created the payment method **outside** `Customer`.
- Then we passed it in the constructor.

WHAT HAPPENED



Experiment 2 and 3

Swap the payment method, and test with a fake one

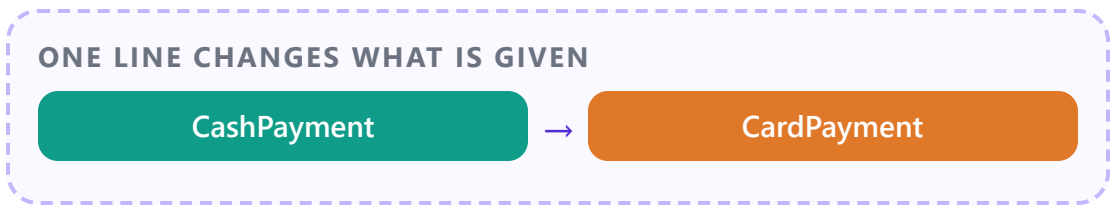
EXPERIMENT 2: SWAP WITH ONE LINE

```
IPaymentMethod payment = new CardPayment();
```

OPEN /BUY AGAIN

```
Paid with card - purchase completed
```

- We never touched `Customer`.
- This is the power of DI: **change what is given, without changing the class**.



EXPERIMENT 3: A FAKE FOR TESTING

```
public class FakePayment : IPaymentMethod
{
    public string Pay() => "Fake payment";
}

Customer customer = new Customer(new FakePayment());
```

RESULT

```
Fake payment - purchase completed
```

- We tested `Customer` without paying for real.
- This is impossible if `Customer` creates `CashPayment` by itself.

Benefits and summary

Why DI is worth it, and what to remember

BENEFITS

Benefit	How
Easy to change	Change the payment method without editing `Customer`
Loose coupling	The customer knows only the interface
Easy to test	Give the customer a fake payment method in a test

SUMMARY

Concept	What it is
Dependency	What a class needs to work
Tight Coupling	A class creates what it needs with `new`, so it is tied to it
Dependency Injection	A class receives what it needs from outside, usually through the constructor
Dependency Inversion	A principle: high-level and low-level modules both depend on abstractions
Interface	Makes the part replaceable

TRY IT YOURSELF

- Add a third payment method `PaypalPayment` and give it to the customer instead of the current one.
- Create `IGreeter`, two classes `ArabicGreeter` and `EnglishGreeter`, and a class `Welcome` that receives `IGreeter` in its constructor. Swap between them with one line.
- Write a `FakeGreeter` and use it with `Welcome`.

REMEMBER THE PICTURE

